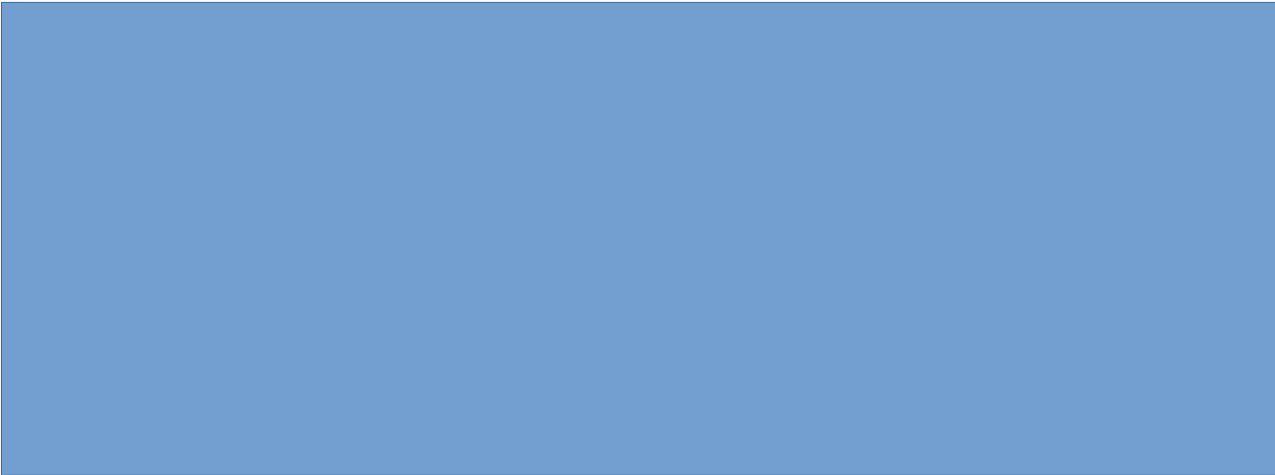
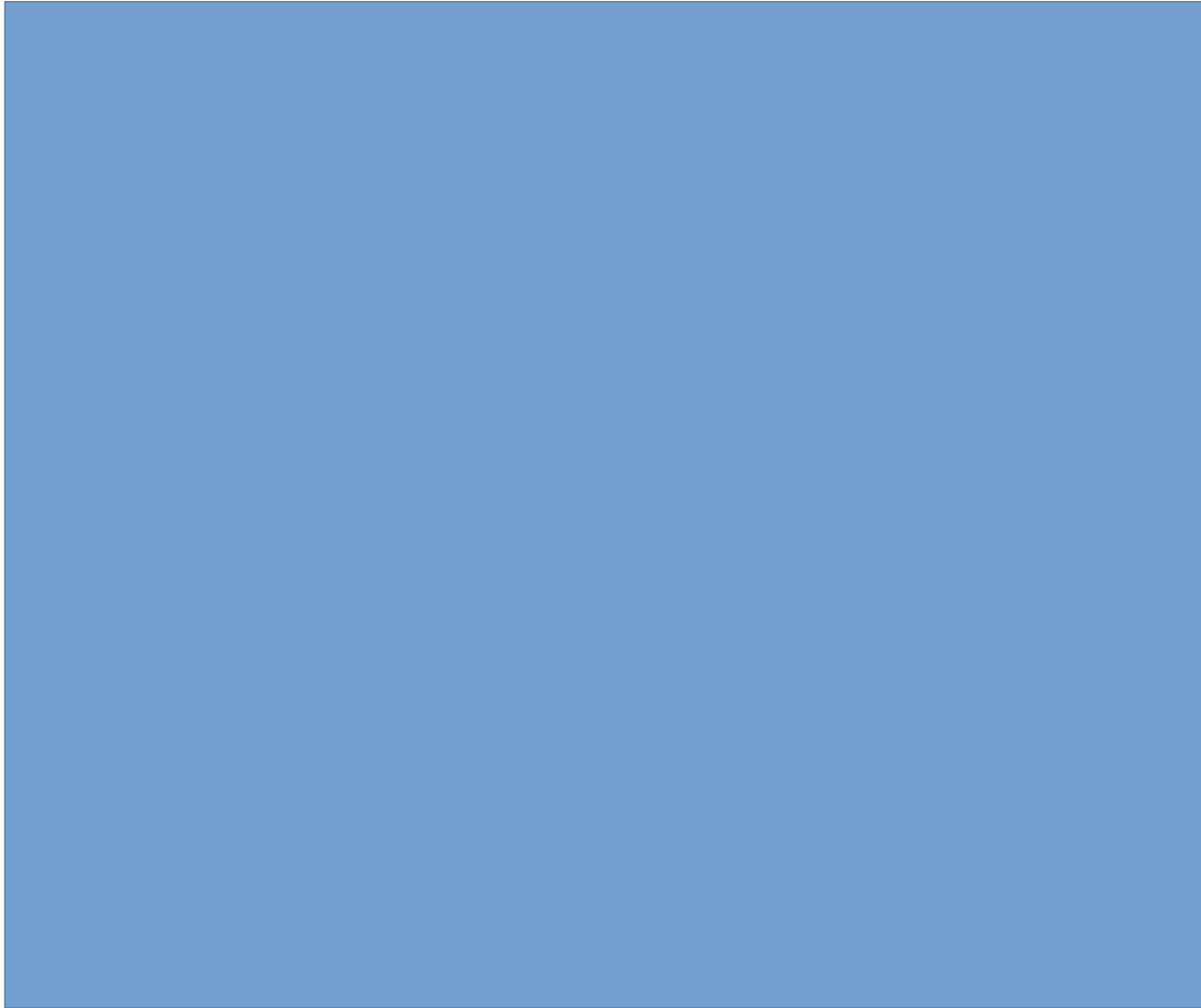




Fruit classification with neural networks



Index

Introduction.....	3
Dataset analysis.....	3
Selection and training.....	4
Model architecture.....	5
Upgrades and specifications on the model architecture.....	5
Model evaluation and results analysis.....	7
Production with streamlit.....	8
Production with Flask server.....	9
Some upgrades for the future.....	10

Introduction

This document will details the process and solution proposals to create a fruit classifier using a neural network. The problem description is the following: It has to automatize the classification of 5 distinct fruits from picture taken from them. A correct classification between bananas, apples, lemons, oranges and pears must be done. The solution must be accesible and usable for final users and software developers.

To achieve it, the classification will be done by a CNN, which will be trained from scratch with a dataset created specifically for this project (a handmade dataset, not from external sources). Once we get a model that meets the requirements and expectations, it will be necessary to:

- Be able to mount an API server from which you can query and classify.
- A web application that will allow users to upload their fruit photos and classify them.

Dataset analisys

The used dataset is a combination of 286 images where 51 are from apples, 58 are from bananas, 62 are from lemons, 58 are from oranges and 57 are from pears.

The images are stored with jpg format, with a resolution of 4032x3024 pixels.

Since Hugging face's dataset library will be used for the dataset's management during the training of CNN, the directory structure is the following:

```
Fruits/train-----Apple
                    |---Banana
                    |---Lemon
                    |---Orange
                    |---Pear
```

The dataset has two possible danger or details to take into account:

- When dividing into groups (train, validation, test), the data of the same individual shouldn't appear repeated in 2 or more groups, because this can lead to data contamination. Applied to images, it would be best if the same photographed individual doesn't appear repeated in two or more of this groups of data, but, since we are looking for a "simple" classifier or just an experimental demo which doesn't need to be extremely precise or effective, it has been considered that it's best to ignore this condition since it requires extra and meticulous efforts and precautions which doesn't really pay off for what it is pretended to get as the final product.
- Between the apple images, they have been shuffle with red and green apples. Looking for that the classifier will still be able to classify them as apples.

Some examples:



Selection and training

Since the goal is to perform inference on images, where the model must detect and classify the fruit appearing in them, a Convolutional Neural Network (CNN) architecture was selected as the foundation for developing and training the model. Unlike standard Artificial Neural Networks (ANNs) that learn from images on a pixel-by-pixel, CNNs generate a feature map that enables the network to learn contours, shapes, colors, textures... which facilitates accurate image classification, even for images that deviate from the norm.

Depending on the context, CNNs can be used to classify grayscale or color images, using systems such as RGB or RGBA. It is a mistake to assume that the best approach for training a CNN is to use images containing the maximum amount of data -such as high-resolution RGBA images- in the hope of achieving superior classification performance. An excess of data diverts the learning attention process toward details that may not be as significant as others, while also considerably increasing training and inference times as well as model complexity. Therefore, it is important to reduce the data to the minimum amount the model actually requires for accurate classification. Since our specific case involves fruit classification, we must consider the defining characteristics of each fruit, primarily their shapes and colors. Consequently, our CNN needs to be capable of recognizing these shapes and colors, making it feasible to use low-quality RGB images.

At first, it was considered to adapt images to a 512x512 pixel resolution, with the idea of making the model be able to learn clearly from textures and patterns on the skin that fruits tend to have. Unfortunately, the hardware resources weren't enough for short time trainings. So the resolution was lowered to 64x64 pixels, which still allows to study shapes and colors in reasonable amounts of times.



Model architecture

As it has been already mention and speaking in terms from the tensorflow framework. The selected architecture will be **CNN**, so **Conv2D** layers will be used to obtains the feature maps of an image, then, for each Conv2D layer, there will be a following **MaxPooling2D** layer, reducing the feature maps's sizes to only process the most important data. From now on, the term "convolutional group" will be applied to refer to the combination of a Conv2D and MaxPooling2D layers. After processing data through convoluational groups, the final feature maps will be processed through **Dense** layers which will give us the classification. The activation function applicable, either for convolutional groups and Dense layers, will be ReLU, since it is an option thoroughly studied and known for a great performace. Depending on the model's necessities, multiple convolutional groups and Dense layers can be applied. The model will return his classification in One-Hot Encoding format through a Dense layer at the end point of the model using the activation function softmax.

Upgrades and specifications on the model architecture

One major problem in neural networks is their tendecies for over-fitting, and CNNs are not the exception. To solve it, some additional techniques has been applied to mitigate over-fitting during training sessions:

- **Dropouts:** Some neurons from the network are intentionally "disable", forcing the network to recalculate their wheights. This common technique is no only applicable to Dense layer, but also to convolutional layers with **SpatialDropout2D** layers.
- **Early Stopping:** When the neural network "stabilize" and no longer can learn the general "concepts", it doesn't automatically stop there, but it starts to learn the specific details from the dataset, leading to over-fitting until all epochs are done. With **Early Stopping**, we can stop it before it happens.

Now, related to other properties and details like hiper-params, one thing to considerate is that the model will process RGB 64x64 pixels images, which is the equivalent of 12288 input params, so the neurnal network will require more capabilities than other neural networks who are trained with classic datasets like MNIST or Fashion (grey scale and 28x28 pixels images, equivalent to 784

input parameters). Bearing that in mind and after some testing, the following architecture was reached:

```
upgraded_model = tf.keras.Sequential([
    layers.Input(shape=(IMAGE_SIZE, IMAGE_SIZE, 3)),

    # Tensorflow data augmentation
    layers.RandomFlip("horizontal_and_vertical"),
    layers.RandomRotation(factor=0.5, fill_mode="reflect"),

    # Convolutional layers
    layers.Conv2D(filters=20, kernel_size=(7, 7), activation="relu"),
    layers.SpatialDropout2D(rate=0.15),
    layers.MaxPooling2D(pool_size=(4, 4)),

    layers.Conv2D(filters=15, kernel_size=(10, 10), activation="relu"),
    layers.SpatialDropout2D(rate=0.15),
    layers.MaxPooling2D(pool_size=(4, 4)),

    layers.Flatten(),

    # Dense layers
    layers.Dense(units=40, activation="relu"),
    layers.Dropout(rate=0.15),
    layers.Dense(units=40, activation="relu"),
    layers.Dropout(rate=0.15),

    layers.Dense(units=len(target_ids), activation="softmax")
])
```

As we can see, two convolutional groups and two Dense layers were required so the model could learn and perform well. Since the images are 64x64px, convolutional groups requires a better “observation capabilities”, which can be achieved adding filters, kernel sizes and, since that contribute to data excess, adjusting pool sizes.

Model evaluation and results analysis

For the model's evaluation, the metrics used for it will be: **Accuracy**, **F1**, **Precision**, **Recall** and **confusion matrixes**. These metrics are often enough to have a good analysis of the models and not just that they "classify well".

The model's results for each data group are:

- Training

```
{'accuracy': 0.8571428571428571,
 'f1': {'Apple': np.float64(0.7333333333333333),
        'Banana': np.float64(0.7605633802816901),
        'Lemon': np.float64(0.9230769230769231),
        'Orange': np.float64(0.972972972972973),
        'Pear': np.float64(0.8529411764705882)},
 'precision': {'Apple': np.float64(0.8461538461538461),
               'Banana': np.float64(0.75),
               'Lemon': np.float64(0.875),
               'Orange': np.float64(0.972972972972973),
               'Pear': np.float64(0.8285714285714286)},
 'recall': {'Apple': np.float64(0.6470588235294118),
            'Banana': np.float64(0.7714285714285715),
            'Lemon': np.float64(0.9767441860465116),
            'Orange': np.float64(0.972972972972973),
            'Pear': np.float64(0.8787878787878788)}}
```

- Validation

```
{'accuracy': 0.8478260869565217,
 'f1': {'Apple': np.float64(0.8571428571428571),
        'Banana': np.float64(0.75),
        'Lemon': np.float64(0.8421052631578947),
        'Orange': np.float64(1.0),
        'Pear': np.float64(0.8)},
 'precision': {'Apple': np.float64(0.8571428571428571),
               'Banana': np.float64(0.75),
               'Lemon': np.float64(0.8),
               'Orange': np.float64(1.0),
               'Pear': np.float64(0.8571428571428571)},
 'recall': {'Apple': np.float64(0.8571428571428571),
            'Banana': np.float64(0.75),
            'Lemon': np.float64(0.8888888888888888),
            'Orange': np.float64(1.0),
            'Pear': np.float64(0.75)}}
```

- Testing or evaluation:

```
{'accuracy': 0.8275862068965517,  
  'f1': {'Apple': np.float64(0.6),  
        'Banana': np.float64(0.782608695652174),  
        'Lemon': np.float64(1.0),  
        'Orange': np.float64(1.0),  
        'Pear': np.float64(0.7741935483870968)},  
  'precision': {'Apple': np.float64(0.6),  
               'Banana': np.float64(0.75),  
               'Lemon': np.float64(1.0),  
               'Orange': np.float64(1.0),  
               'Pear': np.float64(0.8)},  
  'recall': {'Apple': np.float64(0.6),  
            'Banana': np.float64(0.8181818181818182),  
            'Lemon': np.float64(1.0),  
            'Orange': np.float64(1.0),  
            'Pear': np.float64(0.75)}}
```

From the accuracies, we can get a general idea of the model, where the model guess correctly around an 80%. However, if we watch the other metrics, we can see there are some butts.

The model distinguish pretty good between lemons and oranges. From there, bananas and pears follow, where some mistakes are done but at acceptable rates. Lastly, and the most interesting, apple classification is unstable, the f1 values for apples are completely different between each data group. The most probable problem is that this happens because, inside the apples dataset, there are green and red apples mixed together, and our model can distinguish colors. This means that it is highly probable that the model ended up learning that apples has only one skin color, then, in other data groups, the model saw apples with other skin color, which caused the model to get confused. In other words, most of the red or green apples ended up in the train group, and the others in others groups. Which would explain why there is such a gap between these apple results. Also, it is true, that there is a lack of apple images for each color and increasing the dataset might help.

Production with streamlit

You can create an spanish streamlit platform using the resources in the streamlit_app directory, from which you can try the model:

Clasificador de Frutas con TensorFlow

Elige una imagen...

 2026040...7_noMD.jpg 0.8MB +



Imagen cargada

Clasificar

Predicción: **Naranja**

Confianza: 44.95%

▼ Ver salida cruda del modelo

0	1	2	3	4
0.1375	0.0677	0.2878	0.4495	0.0575

Production with Flask server

Also, a Flask server and a Gradio client application were created to try the model:

First, you must install dependencies:

```
pip install -r flask_server/requirements.txt
```

Then, you execute the server script:

- Linux

```
python3 -m flask_server.server
```

- Windows

```
python -m flask_server.server
```

And lastly, the client script:

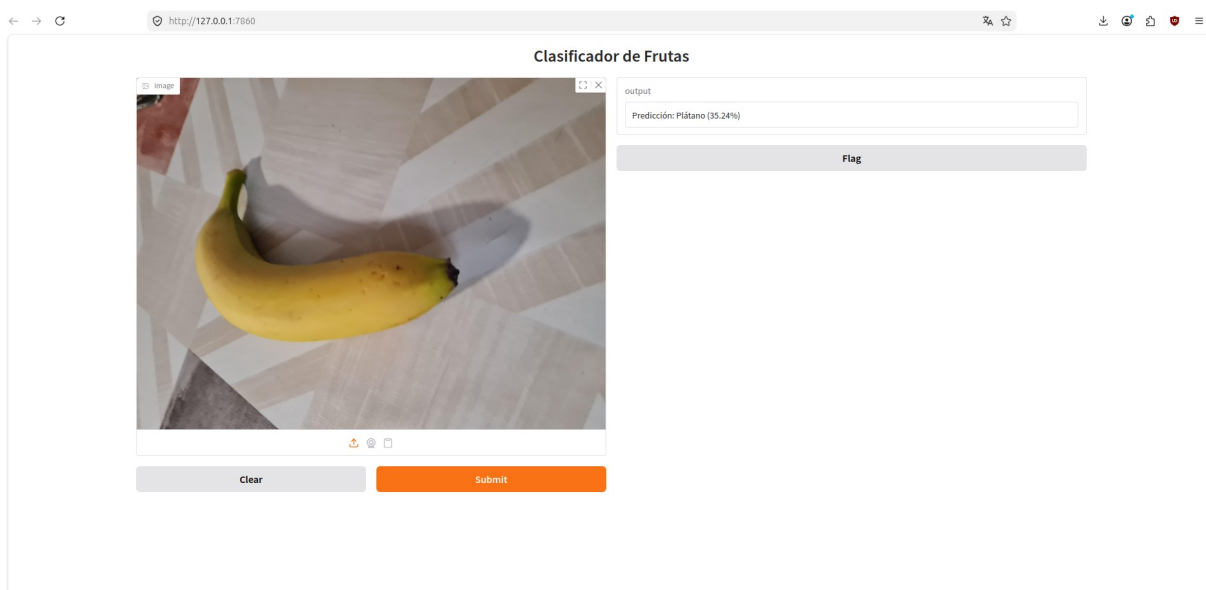
- Linux

```
python3 -m flask_server.client
```

- Windows

```
python -m flask_server.client
```

Now, in your browser, you search for the URL that the Gradio application tells you, by default <http://127.0.0.1:7860>



Some upgrades for the future

As the first objective, it would be advisable to improve those apple classification results. Not only by just increasing the amount of sample images, but also looking for a better dataset with more “diverse” data and not photos of the same individuals.

As the second objective, from the Flask and Streamlit examples, the model classify well, but it “doubts”. In some cases, the probability percentages in the prediction are very low. So it’s also advisable that the model make more secure predictions.